极客时间

Memory 工程

让 AI Agent 拥有持久记忆，用声明式配置驱动工程化

第 1 周 · 第 2 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

回顾：上节课核心要点

- **三层架构** — 规范（你写的）→ Harness（运行时）→ 模型（推理）
- **设计反转** — 人写规范，AI 写代码。80% 时间想清楚要什么
- **实现力高原** — 模型编码能力够用了，瓶颈在规范设计
- **无状态 + 有状态** — 模型是无状态函数，Harness 是有状态系统

极客时间黄佳老师出品

什么是 Memory？— AI 的持久记忆系统

- **会话记忆（短期）** — 当前对话的上下文，关闭即丢失
- **项目记忆（长期）** — 写在文件里的规则和知识，跨会话持久存在
- **Memory 的本质** — 一个纯文本配置文件，每次会话自动加载到 Agent 的提示词中
- **类比** — 就像新员工入职时读的《项目开发手册》，读一次就知道团队规范
- **关键点** — Memory 不是数据库，是声明式的“意图描述文件”

极客时间黄佳老师出品

对比：没有 Memory vs 有 Memory 的 AI

极客时间

没有 Memory 的 Agent

每次会话都是「新员工」
不知道项目用什么框架
不清楚编码规范和命名习惯
可能生成不符合团队风格的代码
需要反复提醒同样的信息
产出质量不稳定，依赖提示词

有 Memory 的 Agent

每次会话都是「老员工」
清楚项目技术栈和架构
遵循团队编码规范
生成的代码风格一致
自动遵守约定，无需重复提醒
产出质量稳定可预期

极客时间黄佳老师出品

各 AI 编程工具的 Memory 文件

极客时间

工具	Memory 文件	位置	格式
Claude Code	CLAUDE.md	项目根目录	Markdown
OpenCode	AGENTS.md	项目根目录	Markdown
Cursor	.cursorrules	项目根目录	纯文本/Markdown
Windsurf	.windsurfrules	项目根目录	纯文本/Markdown
GitHub Copilot	copilot-instructions.md	.github/	Markdown
Cline	.clinerules	项目根目录	纯文本/Markdown

工程思想：声明式配置优先

Declarative Configuration First — 现代 AI Agent 工程化的基石

极客时间黄佳老师出品

命令式 vs 声明式：本质区别

极客时间

命令式 (Imperative)

告诉系统“怎么做”(HOW)
描述执行步骤和流程
执行顺序很重要
例:先建表，再插入数据，再加索引
例: Shell 脚本部署服务器

关注过程 (Process)

声明式 (Declarative)

告诉系统“要什么”(WHAT)
描述期望的最终状态
执行顺序由系统决定
例: 我需要一个有索引的用户表
例: Kubernetes YAML 声明服务

关注结果 (Outcome)

为什么声明式更适合 AI Agent?

极客时间

- **可审计 (Auditable)** — 打开 AGENTS.md 就能看到所有规则，一目了然
- **可版本控制 (Versionable)** — 用 Git 管理，每次改动有记录、可回滚
- **可协作 (Collaborative)** — 团队成员可以 Review、讨论、共同维护规则
- **可迁移 (Portable)** — 换工具换模型，Memory 文件直接复用
- **可组合 (Composable)** — 子目录可以有自己的 AGENTS.md，覆盖或扩展父级规则
- **低耦合 (Decoupled)** — 规则与 Agent 实现分离，互不依赖

极客时间黄佳老师出品

命令式配置 Agent 行为

极客时间

Python

```python
# 命令式: 用代码/脚本控制 Agent 的每一步行为
def configure_agent(agent):
    agent.set_language("python")
    agent.set_style("google")
    agent.add_rule("always_add_docstring", True)
    agent.add_rule("max_function_length", 50)
    agent.add_rule("use_type_hints", True)
    agent.set_test_framework("pytest")
    agent.set_naming_convention("snake_case")
    agent.add_forbidden_pattern("print()")
    agent.add_required_import("from typing import Optional")
    # ... 每增加一条规则就要改代码、重新部署 ...

# 问题:
# 1. 规则散落在代码里，不直观
# 2. 修改规则需要改代码、重新运行
# 3. 非程序员无法参与维护
# 4. 换一个 Agent 框架就要重写
```

声明式配置 (AGENTS.md)

极客时间

AGENTS.md | Markdown

```markdown
# AGENTS.md — 项目 Memory 文件

## 项目概述
本项目是一个 Python Web API 服务，使用 FastAPI 框架。

## 技术栈
- 语言: Python 3.12
- 框架: FastAPI + Uvicorn
- 数据库: PostgreSQL + SQLAlchemy
- 测试: pytest + httpx

## 编码规范
- 遵循 Google Python Style Guide
- 所有函数必须有 docstring
- 函数不超过 50 行
- 使用 type hints
- 变量命名: snake_case
- 禁止裸 print()，使用 logging 模块

## 项目结构
- src/api/      — API 路由
- src/models/   — 数据模型
- src/services/ — 业务逻辑
- tests/        — 测试文件
```

极客时间

声明式配置的核心价值

描述“要什么”而非“怎么做”
让 Agent 自己决定最优执行路径

可审计 · 可版本控制 · 可协作 —— 三位一体的工程化保障

Memory 实战

从零开始为你的项目编写 AGENTS.md

极客时间

极客时间

极客时间

从 Memory 到规范驱动开发（SDD）

极客时间

层级	文件	作用	课节
项目规范	AGENTS.md	项目的技术栈、编码规范、架构约束	本节（第 2 节）
角色规范	agents/*.md	每个 Agent 的身份、权限、职责	第 3 节
能力规范	skills/*/SKILL.md	每个可复用技能的步骤和输入输出	第 4 节

极客时间

AGENTS.md 的 6 个组成部分

- **项目概述** — 一句话说清项目是什么、做什么（让 Agent 建立全局认知）
- **技术栈** — 语言、框架、数据库、测试工具等（防止 Agent 推荐错误的技术）
- **编码规范** — 命名规则、代码风格、禁止项（统一团队代码风格）
- **项目结构** — 目录布局和职责（让 Agent 知道代码该放哪里）
- **工作流程** — 提交规范、分支策略、CI/CD（与团队流程对齐）
- **特殊约束** — 安全要求、性能要求、合规要求等（守住底线红线）

极客时间黄佳老师出品

实战：编写知识库项目的 AGENTS.md

极客时间

AGENTS.md | Markdown

```markdown
# AGENTS.md

## 项目概述
个人知识库管理系统，支持 Markdown 文档的存储、检索和 AI 问答。

## 技术栈
- 前端: Vue 3 + Vite + Element Plus
- 后端: Python 3.12 + FastAPI
- 数据库: SQLite (开发) / PostgreSQL (生产)
- 向量数据库: ChromaDB
- AI 模型: 通过 OpenAI 兼容 API 调用 DeepSeek

## 编码规范
- Python: 遵循 PEP 8，使用 ruff 格式化
- Vue: 使用 Composition API + <script setup> 语法
- 所有 API 端点必须有 Pydantic 请求/响应模型
- 错误处理: 使用自定义异常类，禁止裸 except

## 项目结构
- frontend/src/   — Vue 前端源码
- backend/api/    — FastAPI 路由
- backend/core/   — 核心业务逻辑
- backend/models/ — 数据模型定义
```

编码规范注入：让 Agent 自动遵守团队规则

极客时间

AGENTS.md — 编码规范部分 | Markdown

```
## 编码规范 (详细版)

### 命名规则
- 文件名: kebab-case (如 user-service.py)
- 类名: PascalCase (如 UserService)
- 函数/变量: snake_case (如 get_user_by_id)
- 常量: UPPER_SNAKE_CASE (如 MAX_RETRY_COUNT)
- 私有方法: 前缀下划线 (如 _validate_input)

### 必须遵守
- 每个函数不超过 30 行
- 每个文件不超过 300 行
- 所有公开函数必须有 docstring (Google 风格)
- 所有 API 返回统一格式: {"code": 0, "data": ..., "msg": ""}

### 禁止事项
- 禁止使用 print() 调试，使用 logging
- 禁止 import *
- 禁止在循环中进行数据库查询 (N+1 问题)
- 禁止硬编码密钥或密码
```

上下文管理策略：让 Memory 更高效

- **分层配置** — 根目录 AGENTS.md 放通用规则，子目录放特定规则
- **保持精简** — 控制在 500 行以内，太长反而稀释关键信息
- **优先级明确** — 最重要的规则放最前面，Agent 注意力有衰减
- **定期维护** — 随项目演进更新 Memory，过期规则及时清理
- **团队共建** — Memory 文件纳入 Code Review 流程，团队共同维护

极客时间黄佳老师出品

效果演示：有 Memory vs 无 Memory 的产出对比

极客时间

无 Memory：Agent 自由发挥

使用 Flask 而非 FastAPI (猜错框架)
变量命名 camelCase (不符合 PEP 8)
使用 print() 打印日志
没有 type hints
返回格式不统一
文件放在根目录 (不知道项目结构)

有 Memory：Agent 遵守规范

正确使用 FastAPI + Pydantic
变量命名 snake_case (符合 PEP 8)
使用 logging 模块记录日志
完整的 type hints 和 docstring
统一返回 {"code": 0, "data": ...}
文件放在 backend/api/ 目录下

本节作业

极客时间

任务 1：为你的项目编写 AGENTS.md

选择一个你正在做的项目（或创建一个新项目），按照 6 个组成部分的框架，编写一份完整的 AGENTS.md。至少包含：项目概述、技术栈、编码规范、项目结构。

任务 2：对比实验：有 Memory vs 无 Memory

在 OpenCode 中，分别在有 AGENTS.md 和删除 AGENTS.md 的情况下，给出同样的编程指令（如“写一个用户登录接口”），对比两次产出的代码质量、代码风格和规范遵守程度。截图记录差异。

任务 3：思考题：你的 Memory 遗漏了什么？

在使用 Agent 的过程中，观察它是否产出了不符合你期望的代码。如果有，思考是 AGENTS.md 里缺了什么规则，然后补充上去。

本节小结

极客时间

AGENTS.md = 项目的“入职手册”

Memory 三层：项目规范 → 角色规范 → 能力规范

声明式配置优先：写 What，不写 How

Memory 在上下文压缩后会被重新注入——持久生效的秘密

极客时间

下一节：**Sub-Agents** 角色分工 — 让多个 **Agent** 协作完成复杂任务